



Universidade Federal de Goiás
Instituto de Matemática e Estatística
Bacharelado em Matemática Aplicada e Computacional

Componente Curricular: INF0286 - ALGORITMOS E ESTRUTURAS DE DADOS 1

Turma A: 2025/2

Professora: Dra. Renata Dutra Braga

Revisão para Avaliação 01

Conteúdo para a Avaliação 01 (09/09/25):

- Introdução à disciplina, Revisão de conceitos de Introdução à Computação.
- Registros, Ponteiros e Alocação dinâmica
- Tipos Abstratos de Dados (TADs).

1. Introdução à disciplina e Revisão de conceitos

Ex. 1. Leia dois números, calcule soma e produto, e informe qual é maior.

```
#include <stdio.h>
int main(void){
    float a,b;
    scanf("%f%f",&a,&b);
    printf("Soma: %.2f\nProduto: %.2f\n", a+b,a*b);
    if(a>b) printf("Maior: %.2f\n",a);
    else if(b>a) printf("Maior: %.2f\n",b);
    else printf("São iguais\n");
    return 0;
}
```

Ex. 2. Leia um número real e exiba sua parte inteira e fracionária.

```
#include <stdio.h>
int main(void){
```

```
float n; scanf("%f",&n);
int parteInt = (int)n;
printf("Inteira: %d\nFrac: %.3f\n", parteInt, n - parteInt);
return 0;
}
```

Ex. 3. Leia n inteiros e calcule a média.

```
#include <stdio.h>
int main(void){
    int n,x,soma=0;
    if (scanf("%d",&n)!=1 || n<=0){
        fprintf(stderr,"n invalido\n");
        return 1;
    }
    for(int i=0;i<n;i++){
        scanf("%d",&x);
        soma+=x;
    }
    printf("Média: %.2f\n", (float)soma/n);
    return 0;
}
```

Ex. 4. Qual o erro neste código?

```
float media(int a,int b){ return (a+b)/2; }
```

```
// Falta 2.0 para evitar divisão inteira:
float media(int a,int b){ return (a+b)/2.0f; }
```

2. Registros, Ponteiros e Alocação Dinâmica

Ex. 5. Struct dinâmica: Alocar Aluno com malloc, ler dados e liberar.

```
#include <stdio.h>
#include <stdlib.h>
struct Aluno{ char nome[80]; int matricula; float ira; };
int main(void){
    struct Aluno *p = malloc(sizeof *p);
    scanf("%79s%d%f", p->nome, &p->matricula, &p->ira);
    printf("%s %d %.2f\n", p->nome, p->matricula, p->ira);
    free(p);
    return 0;
}
```

Ex. 6. Função que multiplica dimensões de um struct Retangulo.

```
#include <stdio.h>
struct Retangulo{float w,h};
void redimensionar(struct Retangulo *r, float f){ r->w*=f;r->h*=f;}
int main(void){
    struct Retangulo r={2,3};
    redimensionar(&r,2);
    printf("w=%.1f h=%.1f\n", r.w, r.h);
}
```

Ex. 7. Explique o problema:

```
void f(){ int *p=malloc(100*sizeof(int)); }
```

```
void f(){
    int *p = malloc(100*sizeof *p);
    // uso
    free(p);
}
```

Explicação: Sem free, ocorre memory leak. Reforça importância de liberar memória.

3. Tipos Abstratos de Dados (TADs)

Ex. 8. TAD Triângulo Retângulo — base, altura, área, hipotenusa

Objetivo. TAD para triângulo retângulo definido por base e altura.

Arquivos.

- `triangulo.h`, `triangulo.c`, `main_triangulo.c`

Funções (interface).

- `Triangulo* tri_cria(float base, float altura);`
- `float tri_area(const Triangulo* t); // (b*h)/2`
- `float tri_hip (const Triangulo* t); // sqrt(b^2 + h^2)`
- `void tri_libera(Triangulo* t);`

Regras.

- Rejeite `base < 0` ou `altura < 0` (retorne `NULL`).
- Tipo opaco no `.h`.
- Retorne `-1.0f` em funções numéricas se `t == NULL`.

Cliente (I/O).

- Entrada: `base altura (float)`.
- Saída: `area=...` e `hip=...` com 2 casas.
Ex.: `3 4` → `area=6.00` e `hip=5.00`.

triangulo.h

```
#ifndef TRIANGULO_H
#define TRIANGULO_H

/* Tipo opaco: detalhes da struct ficam apenas no .c */
typedef struct triangulo Triangulo;

/* Cria um triângulo retângulo com base e altura.
   Retorna NULL se valores forem inválidos (negativos). */
Triangulo* tri_cria(float base, float altura);

/* Retorna a área do triângulo ((base*altura)/2).
   Retorna -1.0f se t == NULL. */
```

```

float tri_area(const Triangulo* t);

/* Retorna a hipotenusa (sqrt(b^2 + h^2)).
   Retorna -1.0f se t == NULL. */
float tri_hip(const Triangulo* t);

/* Libera a memória associada ao triângulo. */
void tri_libera(Triangulo* t);

#endif

```

triangulo.c

```

#include "triangulo.h"
#include <stdlib.h>
#include <math.h>

/* Estrutura oculta (tipo opaco) */
struct triangulo {
    float b;
    float h;
};

Triangulo* tri_cria(float base, float altura) {
    if (base < 0.0f || altura < 0.0f) return NULL;
    Triangulo* t = (Triangulo*) malloc(sizeof(Triangulo));
    if (t) {
        t->b = base;
        t->h = altura;
    }
    return t;
}

float tri_area(const Triangulo* t) {
    if (!t) return -1.0f;
    return (t->b * t->h) / 2.0f;
}

float tri_hip(const Triangulo* t) {
    if (!t) return -1.0f;
    return sqrtf(t->b * t->b + t->h * t->h);
}

void tri_libera(Triangulo* t) {
    free(t);
}

```

main_triangulo.c

```
#include <stdio.h>
#include "triangulo.h"

int main(void) {
    float base, altura;
    if (scanf("%f %f", &base, &altura) != 2) {
        fprintf(stderr, "Entrada inválida.\n");
        return 1;
    }

    Triangulo* t = tri_cria(base, altura);
    if (!t) {
        fprintf(stderr, "Valores inválidos (negativos não permitidos).\n");
        return 1;
    }

    printf("area=%.2f\n", tri_area(t));
    printf("hip=%.2f\n", tri_hip(t));

    tri_libera(t);
    return 0;
}
```

Ex. 9. TAD Circulo – raio, área, diâmetro

Implemente circulo.h / circulo.c com as seguintes funções:

```
Circ*  circ_cria(float r);

float  circ_area(const Circ* c);    //  $\pi * r^2$ 

float  circ_diam(const Circ* c);    //  $2 * r$ 

void   circ_libera(Circ* c);
```

Objetivo: Criar um TAD que represente um **círculo** definido pelo **raio**, com operações para calcular **área** e **diâmetro**.

Regras:

- Rejeite valores negativos de raio (retorne NULL).
- Utilize **tipo opaco** no .h (não revele a struct).
- Em funções numéricas, retorne -1.0f caso o ponteiro do círculo seja NULL.

Cliente (I/O esperado):

- Entrada: um valor raio (float).
- Saída: area=... e diam=... com 2 casas decimais.
 - Exemplo: entrada 3 → saída area=28.27 diam=6.00.

```
circulo.h
#ifndef CIRCULO_H
#define CIRCULO_H

/* Tipo opaco exportado */
typedef struct circulo Circ;

/* Cria um círculo com raio r. Retorna NULL se r < 0 (inválido).
*/
Circ* circ_cria(float r);

/* Retorna a área do círculo ( $\pi r^2$ ). Se c==NULL, retorna -1.0f.
*/
float circ_area(const Circ* c);

/* Retorna o diâmetro do círculo (2r). Se c==NULL, retorna -1.0f.
*/
float circ_diam(const Circ* c);

/* (Opcional, mas recomendado): libera o objeto. */
void circ_libera(Circ* c);

#endif /* CIRCULO_H */

circulo.c
#include "circulo.h"
#include <stdlib.h> /* malloc, free */
#include <float.h> /* para valores sentinela, se desejar */
```

```

/* Representação oculta ao cliente (encapsulamento) */
struct circulo {
    float r;
};

/* Definição de PI sem depender de M_PI */
static const float PI = 3.14159265358979323846f;

Circ* circ_cria(float r) {
    if (r < 0.0f) return NULL;          /* raio inválido */
    Circ* c = (Circ*) malloc(sizeof(Circ));
    if (c) c->r = r;
    return c;
}

float circ_area(const Circ* c) {
    if (!c) return -1.0f;
    return PI * c->r * c->r;
}

float circ_diam(const Circ* c) {
    if (!c) return -1.0f;
    return 2.0f * c->r;
}

void circ_libera(Circ* c) {
    free(c);
}

```

main.c (programa cliente)

```

#include <stdio.h>
#include "circulo.h"

int main(void) {
    float r;
    if (scanf("%f", &r) != 1) {
        fprintf(stderr, "Entrada invalida.\n");
        return 1;
    }
    Circ* c = circ_cria(r);
    if (!c) {

```



```
        fprintf(stderr, "Raio invalido (negativo).\n");
        return 1;
    }
    printf("area=%.2f\n",  circ_area(c));
    printf("diam=%.2f\n",  circ_diam(c));
    circ_libera(c);
    return 0;
}
```